

Model Context Protocol (MCP)



Content

- Introduction to Model Context Protocol (MCP)
- Architecture
- Advantages and Limitations
- Real Life Use cases

Website: <https://inttrvu.ai>

Introduction

Website: <https://inttrvu.ai>

What is MCP?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems (Tools).

Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.

Core Idea:

Allows LLMs to dynamically access, interpret, and use **contextual information** beyond their static training data.

Think of MCP like a **USB-C port** for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems (Tools).

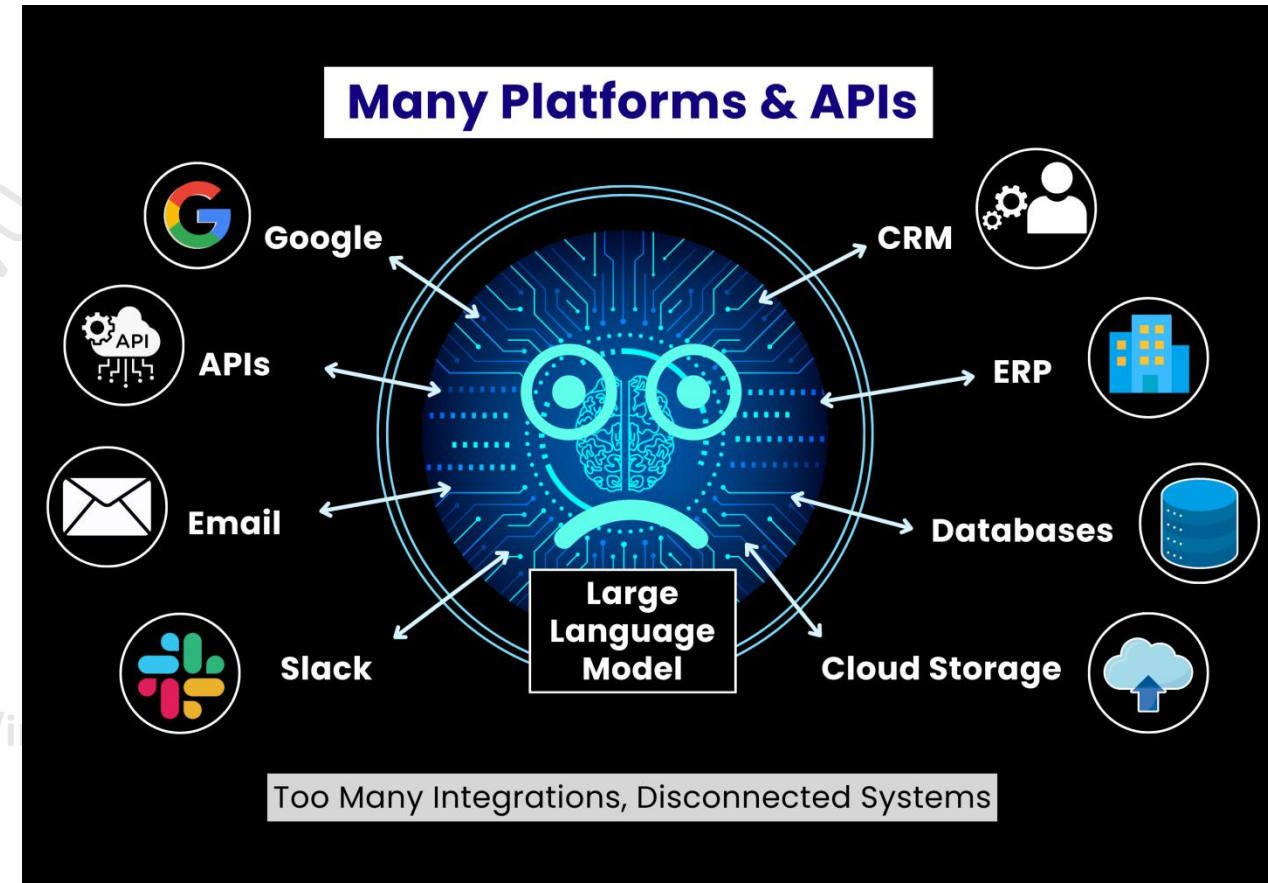


Why MCP?

LLMs Alone Aren't Enough :

- LLMs can understand and reason well, **but they struggle to handle many tools or APIs.**
- As integrations grow, it's **hard for the model to know which tool to use and when.**
- Custom connections make systems **complex, slow to update, and hard to maintain.**

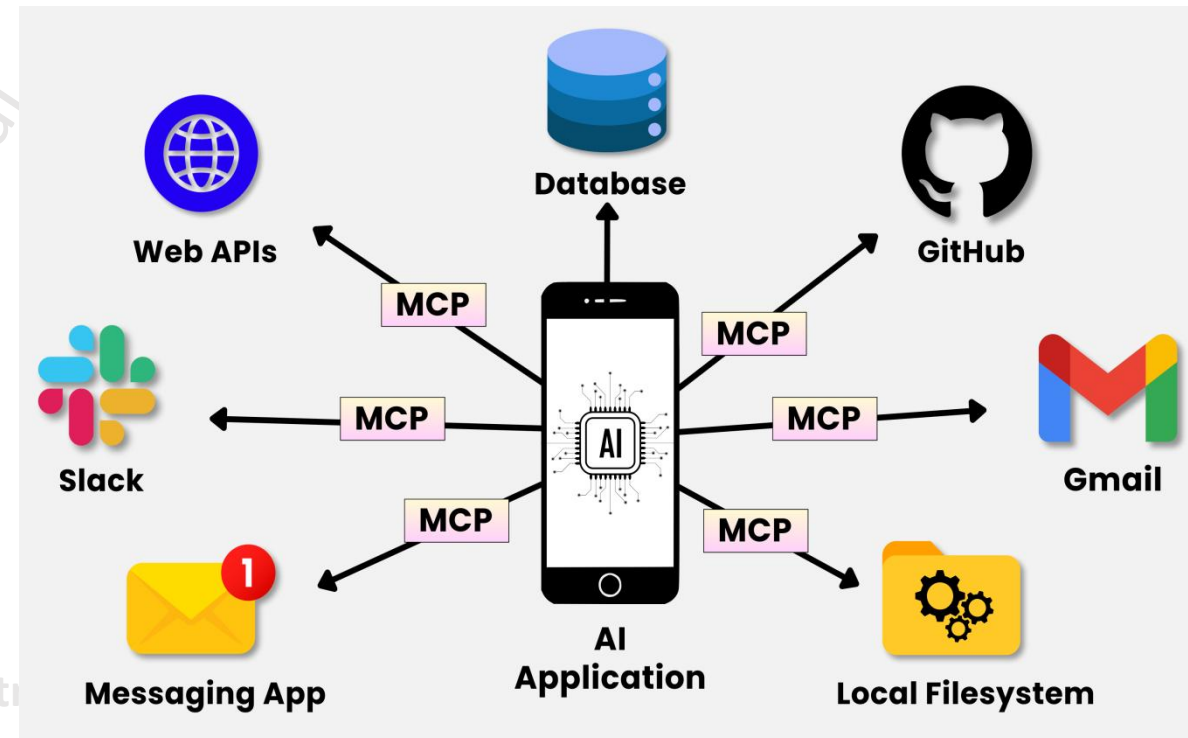
Website: <https://i>



Why MCP?

How MCP Solves This :

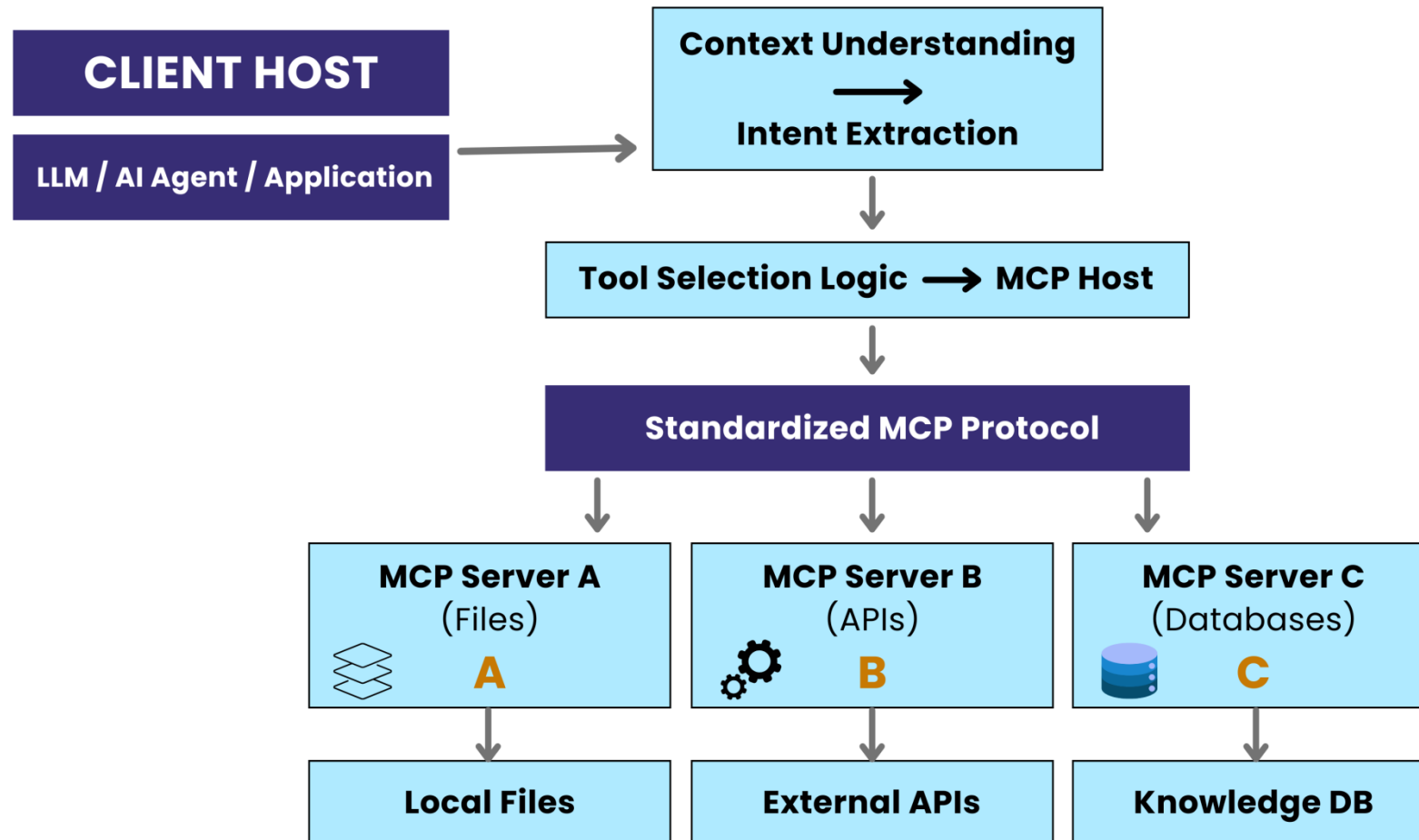
- **Standardized Interface:** One protocol to connect all tools, data sources, and apps.
- **Smart Context Awareness:** MCP helps agents discover and call the right tools automatically.
- **Simplified Development:**
 - *Developers:* Build once, reuse everywhere.
 - *AI Agents:* Instantly gain access to compatible tools and data.
 - *End-Users:* Get smarter AI that can access real data and take meaningful actions.



Architecture

Website: <https://inttrvu.ai>

Architecture



MCP Components

Component	Description	Key Responsibilities	Example / Analogy
Client	The AI model, agent, or application that needs to use external tools or data.	- Sends requests (queries, actions) - Interprets responses from tools	ChatGPT, LangChain Agent
Host	Middleware that manages communication and tool discovery between the Client and external Servers.	- Registers available tools - Manages sessions & routing - Handles authentication	MCP Runtime, AI Gateway
Protocol (MCP)	The standard that defines how the Host and Server communicate.	- JSON-based message exchange - Uses WebSocket or stdio - Ensures interoperability	Like HTTP for AI tools
Server	Provides tools, APIs, or data sources exposed through the MCP standard.	- Executes functions or queries - Returns results in structured format	Database, Calendar API, File System

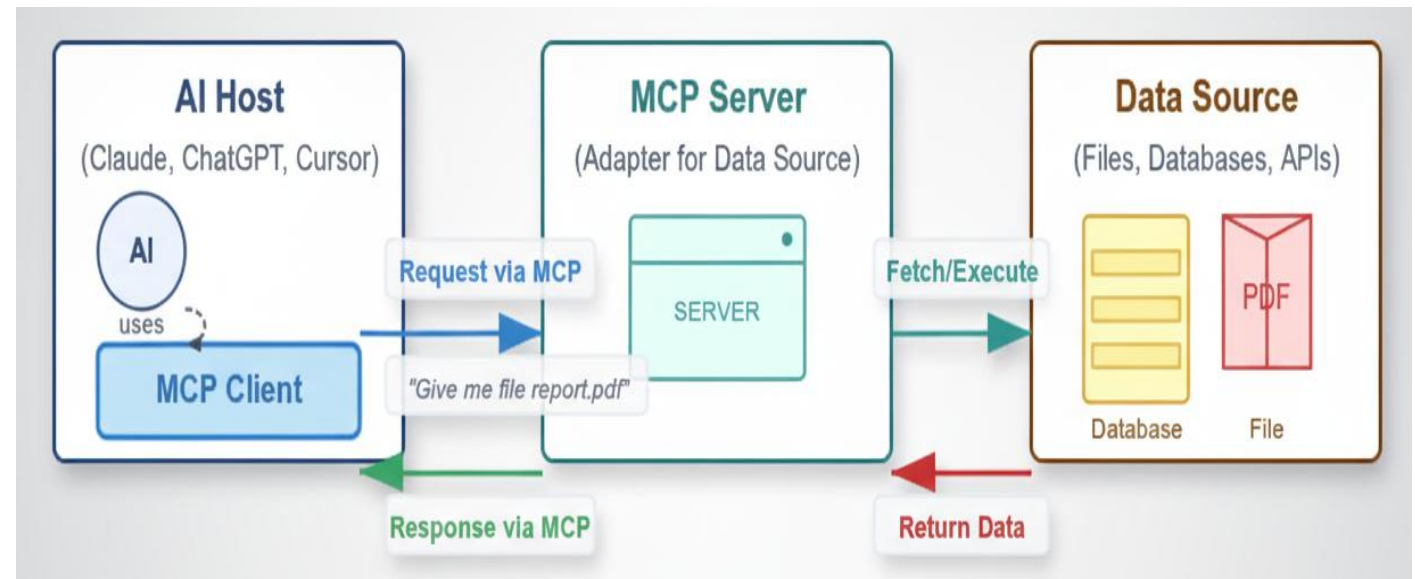
Workflow

1. Agent Request:

The AI agent on the client host receives a user query (e.g., “Fetch sales data for Q3”).

2. MCP Communication:

The host uses the **MCP protocol** to send the request to the right MCP Server based on the context.



Website: <https://inttrvu.ai>

Workflow

3. Server Processing:

The selected MCP Server connects to its backend resource — file system, API, or database — and retrieves or processes the needed data.

4. Response Handling:

The server sends structured results back through the MCP channel to the client host.

5. Agent Response:

The AI agent combines the results with reasoning from the LLM and presents a final, context-aware answer to the user.

Website: <https://inttrvu.ai>

Evolution of Agentic AI with MCP

Feature	Pre-MCP Agentic AI (The Old Way)	Post-MCP Agentic AI (The New Standard)
Tool Integration	Custom & Fragile: You built a separate, proprietary connector for every tool (e.g., custom code just for the GitHub API). This was slow, expensive, and broke easily when APIs changed.	Standardized & Plug-and-Play: Tools expose their capabilities through a single, uniform MCP format. Agents can connect to any MCP-enabled tool instantly—just like plugging a USB-C cable into different devices.
Context Access	Limited & Stale: Agents often relied on context baked into their prompt or memory. Getting real-time data from multiple enterprise systems was a complex, one-off engineering task.	Real-Time & Rich: Agents can securely access the latest, most relevant data and resources (like your company's latest sales figures or a project's current file structure) via the MCP Server. The context is always fresh.
Development Focus	Plumbing: Developers spent most of their time writing custom "plumbing" code to get the AI to <i>talk</i> to the tool.	Intelligence: Developers focus on agentic logic —how the agent plans, reasons, and self-corrects—because the tool communication is already solved by the protocol.

Agno Search Agent Using MCP Servers

1. Client / UI (Claude Desktop, Streamlit, or other front-end)

The user enters a question or search query through any MCP-compatible interface or custom UI.

2. MCP Server (main.py)

Acts as the bridge between the UI and the AI agent — receives the query, identifies the right tool, and routes the request.

Website: <https://inttrvu.ai>

```
from dotenv import load_dotenv
from fastmcp import FastMCP
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools
import logging

load_dotenv()
logging.basicConfig(level=logging.INFO)

mcp = FastMCP("Agno Search MCP")
agent = Agent(model=OpenAIChat(id="gpt-4o"),
              tools=[DuckDuckGoTools()],
              markdown=True, debug_mode=False)

@mcp.tool()
def search(query: str) -> str:
    """Search the web using Agno agent"""
    try:
        res = agent.run(query)
        return getattr(res, "content", str(res))
    except Exception as e:
        logging.exception("Search failed")
        return f"Search failed: {e}"

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

Agno Search Agent Using MCP Servers

3. Agno Agent (Core AI Logic)

Uses the **DuckDuckGo** search tool to fetch real-time information and the **GPT-4o** model to interpret, analyze, and summarize the results.

4. Response Generation

The summarized, context-aware answer is packaged by the MCP server and sent back to the client in a structured format (JSON/text).

Website: <https://inttrvu.ai>

```
from dotenv import load_dotenv
from fastmcp import FastMCP
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools
import logging

load_dotenv()
logging.basicConfig(level=logging.INFO)

mcp = FastMCP("Agno Search MCP")
agent = Agent(model=OpenAIChat(id="gpt-4o"),
              tools=[DuckDuckGoTools()],
              markdown=True, debug_mode=False)

@mcp.tool()
def search(query: str) -> str:
    """Search the web using Agno agent"""
    try:
        res = agent.run(query)
        return getattr(res, "content", str(res))
    except Exception as e:
        logging.exception("Search failed")
        return f"Search failed: {e}"

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

CrewAI Agent Using MCP Servers

1. **Agent** : defines an autonomous CrewAI agent with a role, goal, and background.
2. **mcps field** : lists **MCP endpoints or AMP identifiers** the agent can use.

```
from crewai import Agent

agent = Agent(
    role="Research Analyst",
    goal="Research and analyze information",
    backstory="Expert researcher with access to external tools",
    mcps=[
        "https://mcp.exa.ai/mcp?api_key=your_key",           # External MCP server
        "https://api.weather.com/mcp#get_forecast",         # Specific tool from server
        "crewai-amp:financial-data",                         # CrewAI AMP marketplace
        "crewai-amp:research-tools#pubmed_search"           # Specific AMP tool
    ]
)
```

CrewAI Agent Using MCP Servers

- Each entry is a **discoverable tool source** following the **Model Context Protocol**.
- URLs may point to external APIs, local MCP servers, or marketplace tools.
- CrewAI automatically connects to each MCP server, **discovers available tools**, and makes them callable within the agent's workflow.

```
from crewai import Agent

agent = Agent(
    role="Research Analyst",
    goal="Research and analyze information",
    backstory="Expert researcher with access to external tools",
    mcps=[
        "https://mcp.exa.ai/mcp?api_key=your_key",           # External MCP server
        "https://api.weather.com/mcp#get_forecast",         # Specific tool from server
        "crewai-amp:financial-data",                         # CrewAI AMP marketplace
        "crewai-amp:research-tools#pubmed_search"           # Specific AMP tool
    ]
)
```

Website: <https://inttrvu.ai>

Advantages and Limitations

Website: <https://inttrvu.ai>

Advantages

1. Extensible and Modular

- Components (model, context server, tools) can be added or replaced **independently**.
- Enables easy scalability for enterprise and research use.

2. Secure and Standardized

- Operates via **controlled, permission-based** access.
- Provides a **uniform interface** for LLM-to-tool communication.
- Reduces the risk of data leakage or unsafe execution.

3. Real-Time Tool Integration

- LLMs can query **live databases, APIs, or documents** in real time.
- Enables **dynamic responses** instead of relying on static training data.

Website: <https://inttrvu.ai>

4. Improves Accuracy and Relevance

- Reduces hallucinations by grounding outputs in **verified, current data**.
- Produces **context-aware, fact-based** answers tailored to user needs.

Advantages

Modular	Secure	Real-Time	Accurate
Plug-and-play components	Permission-based access	Live access to APIs/data	Reduces hallucination
Replace tools/models easily	Reduces risk, standard I/O	Dynamic, context-aware replies	Fact-based, grounded answers

website: <https://inttrvu.ai>

Limitations

Disadvantage	Description
Setup & Compatibility	Requires setup + supported clients only
Early Adoption	Limited community tools, evolving docs
Performance in Sandboxes	Restricted access & speed in isolated environments

Real World Use Cases

Website: <https://inttrvu.ai>

Real World Use Cases

Use Case	Description	Example Tools / Systems	Key Benefits
Enterprise Knowledge Assistants	Connect LLMs to internal company databases and document repositories for real-time Q&A.	SharePoint, Confluence, Notion, internal SQL servers	Instant access to organizational knowledge; secure and context-aware responses.
Financial Data Query Bots	AI agents fetch live market, transaction, or portfolio data for analysis and reporting.	Bloomberg API, Excel Sheets, SQL Databases	Accurate financial insights; faster reporting; reduced manual work.
Internal Automation Tools	MCP enables LLMs to automate repetitive workflows across business apps.	Jira, GitHub, Slack, ServiceNow	Task automation; reduced human intervention; faster operations.
Customer Support Integration	AI agents pull customer data and ticket info from CRM or helpdesk systems.	Zendesk, Salesforce, Freshdesk	Personalized, faster, and accurate customer service responses.

Real World Use Cases

1. Standardization Across AI Ecosystems

- Enables consistent model–tool interaction across providers (OpenAI, Anthropic, Google).
- Promotes a shared protocol for AI infrastructure.

2. Agent Framework Integration

- Empowers agentic systems to securely access tools via MCP.
- Supports autonomous workflows with fine-grained context control.

3. Cross-Platform Interoperability

- One MCP-compatible tool can be reused across models and platforms.
- Simplifies enterprise integration and multi-model deployments.

Thank You !

Website: <https://inttrvu.ai>